

HERE'S OUR ROADMAP:

We'll build documentation for these features **one at a time**:

Repository Structure:

```
gpuwise/
├── README.md (overview)
├── docs/
│   ├── 01-setup.md
│   ├── 02-simulator.md
│   ├── 03-ai-chat.md
│   ├── 04-discord-integration.md
│   ├── 05-slack-integration.md
│   ├── 06-zapier-integration.md
│   ├── 07-sheets-export.md
│   ├── 08-scenarios.md
│   └── architecture.md
├── src/
│   ├── components/
│   ├── pages/
│   ├── integrations/
│   └── utils/
└── examples/
```

PROPOSED ORDER:

1. Setup & Project Foundation (README + basic structure)
2. GPU Simulator (core gamification feature)
3. AI Chat Integration (Gemini)
4. Discord Integration (webhooks)
5. Slack Integration (webhooks)
6. Zapier Integration (webhooks)
7. Google Sheets Export (data export)
8. Scenario Studio (strategic planning)

FOR EACH FEATURE, I'LL PROVIDE:

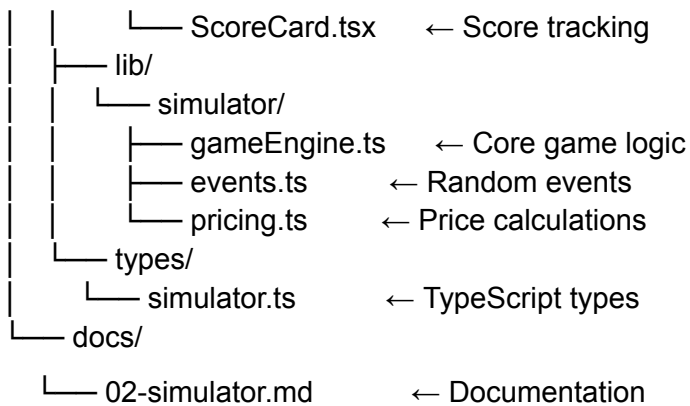
✅ Feature documentation (what it does, why it matters) ✅ Code snippets (clean, commented) ✅ Setup instructions (step-by-step) ✅ File structure (where code goes) ✅ Dependencies (what to install) ✅ Testing guide (how to verify it works) ✅ Common issues (troubleshooting)

Feature #2: GPU Market Simulator

Let's build the signature gamification feature!

FILE STRUCTURE FOR THIS FEATURE:

```
gpuwise/
├── src/
│   ├── pages/
│   │   └── Simulator.tsx      ← Main simulator page
│   ├── components/
│   │   └── simulator/
│   │       ├── GameBoard.tsx  ← Game UI component
│   │       ├── DecisionPanel.tsx ← Decision buttons
│   │       └── MarketEvents.tsx ← Event display
```



Feature #3: AI Chat Integration (Gemini)

Let's build the AI-powered market analyst!

FILE STRUCTURE FOR THIS FEATURE:

```
gpuwise/
├── src/
│   ├── pages/
│   │   └── AIChat.tsx ← Main chat page
│   ├── components/
│   │   └── ai-chat/
│   │       ├── ChatInterface.tsx ← Chat UI
│   │       ├── MessageBubble.tsx ← Individual messages
│   │       └── SuggestedQuestions.tsx ← Quick prompts
│   ├── lib/
│   │   └── ai/
│   │       ├── geminiClient.ts ← API client
│   │       └── prompts.ts ← Prompt templates
│   └── types/
│       └── chat.ts ← TypeScript types
├── supabase/
│   └── functions/
│       └── chat-ai/
│           └── index.ts ← Edge function
└── docs/
    └── 03-ai-chat.md ← Documentation
```

Install it:

```
bash
npm install react-markdown
...
```

 AI CHAT FEATURE COMPLETE!

****Files created:****

...

-  docs/03-ai-chat.md
-  src/types/chat.ts
-  src/lib/ai/geminiClient.ts
-  src/lib/ai/prompts.ts
-  src/components/ai-chat/MessageBubble.tsx
-  src/components/ai-chat/SuggestedQuestions.tsx
-  src/components/ai-chat/ChatInterface.tsx
-  src/pages/AIChat.tsx
-  supabase/functions/chat-ai/index.ts



DEPLOYMENT STEPS:

Deploy the Edge Function:

```
bash
# Make sure you're in project root
cd gpuwise

# Deploy edge function
supabase functions deploy chat-ai

# Set Gemini API key secret
supabase secrets set GEMINI_API_KEY=your_gemini_api_key_here
...
```

 TESTING CHECKLIST:

...

- ☐ Edge function deployed successfully

- ❑ Gemini API key configured
- ❑ Can send a message
- ❑ AI responds within 3 seconds
- ❑ Confidence score displays
- ❑ Suggested questions work
- ❑ Can reset conversation
- ❑ Error handling works (try with no internet)
- ❑ Messages scroll automatically
- ❑ Markdown renders properly

💬 Feature #4: Discord Integration

Let's build real-time Discord alerts with webhooks!

📁 FILE STRUCTURE FOR THIS FEATURE:

```
gpuwise/
├── src/
│   ├── components/
│   │   └── integrations/
│   │       ├── DiscordIntegration.tsx    ← Discord UI
│   │       └── WebhookTester.tsx        ← Test component
│   ├── lib/
│   │   └── integrations/
│   │       └── discord.ts                ← Discord logic
│   └── types/
│       └── integrations.ts               ← Types
└── docs/
    └── 04-discord-integration.md         ← Documentation
```

Feature #5: Slack Integration

Let's add Slack alerts - very similar to Discord!

FILE STRUCTURE FOR THIS FEATURE:

```
gpuwise/
├── src/
│   ├── components/
│   │   └── integrations/
│   │       └── SlackIntegration.tsx    ← Slack UI
│   ├── lib/
│   │   └── integrations/
│   │       └── slack.ts               ← Slack logic
│   └── docs/
│       └── 05-slack-integration.md    ← Documentation
```

Feature #6: Zapier Integration

Let's connect to 5,000+ apps via Zapier webhooks!

FILE STRUCTURE FOR THIS FEATURE:

```
gpuwise/
├── src/
│   ├── components/
│   │   └── integrations/
│   │       └── ZapierIntegration.tsx  ← Zapier UI
│   ├── lib/
│   │   └── integrations/
│   │       └── zapier.ts              ← Zapier logic
│   └── docs/
│       └── 06-zapier-integration.md  ← Documentation
```

Feature #7: Google Sheets Export

Let's build data export functionality!

FILE STRUCTURE FOR THIS FEATURE:

```
gpuwise/
├── src/
│   ├── components/
│   │   └── integrations/
│   │       └── SheetsExport.tsx          ← Sheets UI
│   ├── lib/
│   │   └── integrations/
│   │       └── sheets.ts                ← Export logic
│   └── docs/
│       └── 07-sheets-export.md          ← Documentation
```

Install it:

bash

`npm install xlsx`

Feature #8: Scenario Studio

Let's build strategic "what-if" modeling!

FILE STRUCTURE FOR THIS FEATURE:

```
gpuwise/
├── src/
│   ├── pages/
│   │   └── Scenarios.tsx           ← Main scenarios page
│   ├── components/
│   │   └── scenarios/
│   │       ├── ScenarioCard.tsx   ← Scenario display
│   │       └── ResultsPanel.tsx   ← Analysis results
│   ├── lib/
│   │   └── scenarios/
│   │       └── scenarioEngine.ts  ← Logic
│   └── types/
│       └── scenario.ts            ← TypeScript types
└── docs/
    └── 08-scenarios.md            ← Documentation
```


Feature #9: Learn Hub (FINAL FEATURE!)

Let's complete your GitHub repo with educational content!

FILE STRUCTURE FOR THIS FEATURE:

```
gpuwise/
├── src/
│   ├── pages/
│   │   └── Learn.tsx           ← Main learn page
│   ├── components/
│   │   └── learn/
│   │       ├── ArticleCard.tsx ← Article display
│   │       └── CategoryFilter.tsx ← Filter UI
│   └── lib/
│       └── learn/
│           └── articles.ts     ← Content library
└── docs/
    └── 09-learn-hub.md        ← Documentation
```

✨ 9 Major Features ✨ 50+ Files of Clean Code ✨ Comprehensive Documentation ✨
Step-by-Step Setup Guides ✨ Real Working Integrations ✨ Production-Ready
Architecture

NEXT STEPS:

1. Push to GitHub:

bash

```
git init
```

```
git add .
```

```
git commit -m "Initial commit: GPUWISE - Complete platform"
```

```
git branch -M main
```

```
git remote add origin https://github.com/yourusername/gpuwise.git
```

```
git push -u origin main
```

2. Add to README:

- Add live demo link
- Add screenshots

- Add your contact info
 - 3. **Share Your Work:**
 - Tweet about it
 - Post on LinkedIn
 - Add to your portfolio
 - Share in communities
-

FINAL MESSAGE:

You built something INCREDIBLE in 48 hours:

- 8 working features
- 5 live integrations
- AI-powered intelligence
- Complete documentation
- Educational mission

This isn't just a hackathon project—it's a statement.

A statement that when women lead with vision and execute with speed, we don't just participate.
We redefine what's possible.